Q1. Consider the following grammar and construct LR(0) parsing table.
CODE:

```
#include <iostream>
#include <vector>
#include <string>
#include <map>
#include <set>
#include <iomanip>

using namespace std;

struct Rule {
    string lhs;
    vector<string> rhs;
};

struct Item {
    int rule_idx;
    int dot_pos;
    bool operator<(const Item& other) const {
        if (rule_idx != other.rule_idx) return rule_idx <
other.rule_idx;
        return dot_pos < other.dot_pos;
    }
    bool operator==(const Item& other) const {
        return rule_idx == other.rule_idx && dot_pos ==
other.dot_pos;
    }
};

vector<Rule> rules = {
    {"S'", {"S"}},      // S' -> S
    {"S", {"A"}},       // S -> A
    {"A", {"A", "b"}},  // A -> Ab
    {"A", {"a"}}        // A -> a
};

set<Item> closure(set<Item> I) {
    set<Item> J = I;
    bool changed = true;
    while (changed) {
        changed = false;
        set<Item> temp = J;
        for (auto item : temp) {
            if (item.dot_pos < rules[item.rule_idx].rhs.size()) {
                string next_sym =
rules[item.rule_idx].rhs[item.dot_pos];
                for (size_t i = 0; i < rules.size(); ++i) {
                    if (rules[i].lhs == next_sym) {
                        Item new_item = {(int)i, 0};
                        if (J.find(new_item) == J.end()) {
                            J.insert(new_item);
                            changed = true;
                        }
                    }
                }
            }
        }
    }
```

```
    return J;
}

set<Item> goto_state(set<Item> I, string X) {
    set<Item> next_I;
    for (auto item : I) {
        if (item.dot_pos < rules[item.rule_idx].rhs.size()) {
            if (rules[item.rule_idx].rhs[item.dot_pos] == X) {
                next_I.insert({item.rule_idx, item.dot_pos + 1});
            }
        }
    }
    return closure(next_I);
}

int main() {
    vector<set<Item>> states;
    map<pair<int, string>, int> transitions;

    set<Item> I0 = closure({{0, 0}});
    states.push_back(I0);

    bool changed = true;
    while (changed) {
        changed = false;
        for (size_t i = 0; i < states.size(); ++i) {
            set<string> symbols;
            for (auto item : states[i]) {
                if (item.dot_pos <
rules[item.rule_idx].rhs.size()) {

symbols.insert(rules[item.rule_idx].rhs[item.dot_pos]);
                }
            }
            for (auto X : symbols) {
                set<Item> next_I = goto_state(states[i], X);
                if (!next_I.empty()) {
                    int state_idx = -1;
                    for (size_t j = 0; j < states.size(); ++j) {
                        if (states[j] == next_I) {
                            state_idx = j;
                            break;
                        }
                    }
                    if (state_idx == -1) {
                        states.push_back(next_I);
                        state_idx = states.size() - 1;
                        changed = true;
                    }
                    transitions[{(int)i, X}] = state_idx;
                }
            }
        }
    }
    map<int, map<string, string>> action_table;
    map<int, map<string, int>> goto_table;
    for (size_t i = 0; i < states.size(); ++i) {
        goto_table[i]["S"] = -1;
        goto_table[i]["A"] = -1;
    }
```

```
    for (size_t i = 0; i < states.size(); ++i) {
        for (auto item : states[i]) {
            if (item.dot_pos < rules[item.rule_idx].rhs.size()) {
                string a = rules[item.rule_idx].rhs[item.dot_pos];
                if (a == "a" || a == "b") {
                    action_table[i][a] = "s" +
to_string(transitions[{i, a}]);
                }
            }
        }
        for (auto item : states[i]) {
            if (item.dot_pos == rules[item.rule_idx].rhs.size()) {
                if (item.rule_idx == 0) {
                    action_table[i]["$"] = "Acc";
                } else {
                    vector<string> terms = {"a", "b", "$"};
                    for (auto t : terms) {
                        if (action_table[i][t].empty()) {
                            action_table[i][t] = "r" +
to_string(item.rule_idx);
                        } else {
                            action_table[i][t] += "/r" +
to_string(item.rule_idx);
                        }
                    }
                }
            }
        }
        for (auto nt : {"S", "A"}) {
            if (transitions.find({i, nt}) != transitions.end()) {
                goto_table[i][nt] = transitions[{i, nt}];
            }
        }
    }
    cout << "LR(0) Parsing Table:\n";
    cout << left << setw(10) << "State"
      << setw(10) << "a" << setw(15)
      << "b" << setw(10) << "$"
      << setw(10) << "S" << setw(10)
      << "A" << "\n";
    cout << string(65, '-') << "\n";
    for (size_t i = 0; i < states.size(); ++i) {
        cout << left << setw(10) << i
             << setw(10) << (action_table[i]["a"].empty() ? "" :
action_table[i]["a"])
             << setw(15) << (action_table[i]["b"].empty() ? "" :
action_table[i]["b"])
             << setw(10) << (action_table[i]["$"].empty() ? "" :
action_table[i]["$"])
             << setw(10) << (goto_table[i]["S"] == -1 ? "" :
to_string(goto_table[i]["S"]))
             << setw(10) << (goto_table[i]["A"] == -1 ? "" :
to_string(goto_table[i]["A"]))
             << "\n";
    }
    cout << "\nTrace for input string 'abb':\n";
    vector<string> input = {"a", "b", "b", "$"};
    vector<int> states_stack = {0};
    vector<string> sym_stack;
    int ptr = 0;
```

```
    cout << left << setw(20) << "State Stack" << " | "
     << setw(20) << "Symbol Stack"
     << " | " << setw(15) << "Input"
     << " | Action\n";
    cout << string(85, '-') << "\n";

    while (true) {
        int state = states_stack.back();
        string sym = input[ptr];

        string s_stack_str = "";
        for (auto s : states_stack) s_stack_str += to_string(s) +
" ";

        string sym_stack_str = "";
        for (auto s : sym_stack) sym_stack_str += s + " ";

        string input_str = "";
        for (size_t i = ptr; i < input.size(); ++i) input_str +=
input[i];

        string act = action_table[state][sym];
        if (act.empty()) {
            cout << left << setw(20) << s_stack_str << " | "
            << setw(20) << sym_stack_str << " | "
            << setw(15) << input_str << " | Error\n";
            break;
        }

        string actual_act = act;
        if (act.find('/') != string::npos) {
            actual_act = act.substr(0, act.find('/'));
        }

        if (actual_act[0] == 's') {
            cout << left << setw(20) << s_stack_str << " | "
            << setw(20) << sym_stack_str << " | "
            << setw(15) << input_str << " | Shift "
            << actual_act.substr(1) << "\n";

            states_stack.push_back(stoi(actual_act.substr(1)));
            sym_stack.push_back(sym);
            ptr++;
        } else if (actual_act[0] == 'r') {
            int r_idx = stoi(actual_act.substr(1));
            Rule r = rules[r_idx];
            cout << left << setw(20)
            << s_stack_str << " | "
            << setw(20) << sym_stack_str << " | "
            << setw(15) << input_str << " | Reduce "
            << r_idx << " (" << r.lhs << " -> ";

            for (auto rs : r.rhs) cout << rs;
            cout << ")\n";
            for (size_t i = 0; i < r.rhs.size(); ++i) {
                states_stack.pop_back();
                sym_stack.pop_back();
            }
            int top_state = states_stack.back();
```

```
            sym_stack.push_back(r.lhs);
            states_stack.push_back(goto_table[top_state][r.lhs]);
        } else if (actual_act == "Acc") {
            cout << left << setw(20)
                << s_stack_str
                << " | " << setw(20)
                << sym_stack_str << " | " << setw(15)
                << input_str << " | Accept\n";
            break;
        }
    }

    return 0;
}
```

OUTPUT:

```
student@nitcse-OptiPlex-7000:~/24CS8017/LAB5$ g++ lr0_parser.cpp -o lr0
student@nitcse-OptiPlex-7000:~/24CS8017/LAB5$ ./lr0
LR(0) Parsing Table:
State     a         b             $         S         A
-----------------------------------------------------------------
0         s3                                2         1
1         r1        s4/r1         r1
2                                 Acc
3         r3        r3            r3
4         r2        r2            r2

Trace for input string 'abb':
State Stack          | Symbol Stack          | Input           | Action
-------------------------------------------------------------------------------------
0                    |                       | abb$            | Shift 3
0 3                  | a                     | bb$             | Reduce 3 (A -> a)
0 1                  | A                     | bb$             | Shift 4
0 1 4                | A b                   | b$              | Reduce 2 (A -> Ab)
0 1                  | A                     | b$              | Shift 4
0 1 4                | A b                   | $               | Reduce 2 (A -> Ab)
0 1                  | A                     | $               | Reduce 1 (S -> A)
0 2                  | S                     | $               | Accept
```